A direct route to internal consistency of PTS-rk

October 24, 2026

Abstract

The goal is an actual original-PTS inhabitant of a resource-relativized consistency statement, without using $H_{<\omega}$ or Elementary Arithmetic as intermediate proof theories. The development already supplies faithful certificate codes, source-rank compression, an internal rejection proof for every externally bounded raw-code fragment, and fixed internal dictionary and comparison operations. Independent local source graphs now prove source-to-operation agreement externally, and one fixed internal term proves residual execution agreement for arbitrary represented execution witnesses. The combinatory development supplies internal soundness and proof transport for arbitrary inhabitants of independent indexed Horn proof types. A generic Horn family now binds fragment and clause data internally; a rank-two-type finite table returns faithful raw syntax without generation evidence. Neither constructs the source-rule interpretation for arbitrary represented budgets. That bridge, with source, endpoint and resource provenance, remains open. Reusing Horn elimination can replace a separate internal normalization/preservation argument. It is not a claim of internal consistency.

1 Target and proof obligations

Use the original annotated syntax, Typing.typ, and Ranks.RankSorts throughout. In particular,

$$U_i : U_0, \qquad \mathsf{rank}(\Pi x : A.B) = \max(a+1, b) \quad (A : U_a,\ B : U_b), \qquad \bot = \Pi X : U_0.X : U_1.$$

There is no cumulativity, eta rule, new primitive, or assumed soundness principle. A Rocq theorem that no native term inhabits $\bot$ is an external result; the target here is a closed PTS term of a specified consistency type.

Let D be a fixed carrier of complete proof codes and B a fixed carrier of resources. Codes include contexts, annotations, binary ranks, conversion instructions and any supplied computation witnesses. Define independently:

- $\mathsf{Acc}_\bot(q)$: the source certificate passes its local rule checks and concludes a closed term at the literal syntactic type $\bot$;
- $\mathsf{Res}(q, b)$: structural generation, envelopes and the operation capacities needed for the proof.

Acceptance must not contain native typing or semantic soundness as a field. Resources must not assert rejection, validity of a source model, or existence of a normalizing history. These are conclusions to derive. Standard rejected and malformed codes must also receive resources.

The main remaining theorem is an actual term

$$\mathsf{bs} : \Pi q : D.\Pi b : B.\, \mathsf{Res}(q, b) \to \mathsf{Acc}_\bot(q) \to \bot. \qquad \text{(BS)}$$

All its carriers and quantified types must use one fixed finite collection of ranks, independently of represented source ranks, proof lengths and budgets.

1.1 Final resource elimination

The following continuation package avoids demanding a uniform budget selector:

$$\mathsf{Adm}(q) := \Pi Z : U_1.\, (\Pi b : B.\mathsf{Res}(q,b) \to Z) \to Z,$$
$$\mathsf{Cons}_{\mathrm{dir}} := \Pi q : D.\mathsf{Adm}(q) \to \mathsf{Acc}_{\perp}(q) \to \perp.$$

Once (BS) is constructed, the proposed final term is

$$\lambda q\, a\, h.\ a \perp (\lambda b\, r.\mathsf{bs}\, q\, b\, r\, h).$$

Explicit resources introduce admission by $\lambda Z\, k.k\, b\, r$. For $D : U_d$, $B : U_e$, $\mathsf{Res}(q,b) : U_r$ and $\mathsf{Acc}_{\perp}(q) : U_s$, the calculated formation ranks are $\max(e+2, r+2)$ for $\mathsf{Adm}(q)$ and $\max(d+1, e+3, r+3, s+1)$ for $\mathsf{Cons}_{\mathrm{dir}}$. These final builders and their original formation derivations remain obligations, not consequences of the displayed notation.

Adequacy has three separate requirements: admission of every standard raw code, actual internal acceptance witnesses for every accepted standard judgment, and completeness of the source certificates for original typing. The third is already proved externally. The first two must not be obtained by eliminating the externally impossible Bottom case.

2 Completed foundations and their exact scope

The library `PTSrk.SelfConsistency` imports the original kernel and shared annotated assembler/checker, not an arithmetic interpretation or set model. Its detailed module inventory is in `README.md`; verification commands and assumptions are in `EVIDENCE.md`.

- **Certificates and serialization:** an independent six-rule checker, original-typing adequacy and faithful annotated raw codes, proved externally.
- **Rank compression:** complete inputs are relabeled; checking and Bottom are preserved; labels are bounded by serialized size. This is not internal all-source soundness.
- **Syntactic operations:** fixed-rank lookup, histories, scope, renaming and annotated substitution relations with standard witnesses. Scope is not dependent typing preservation.
- **Finite Bottom rejection:** rank-four internal rejection at every external complete-code size bound, as an externally generated schema.
- **Dictionaries:** one rank-five-type polymorphic consumer for represented lists and independent membership evidence. Rejecting branch packages are still externally supplied.
- **Residual execution:** actual structural comparison and conjunction with exact native-checker output on compiled standard inputs. Source compilation remains external.

2.1 Why the finite schema is already a genuine internal result

For an external bound b, enumerate all raw codes

$$Q_b = \{q \mid \mathsf{codeSize}(q) \le b\}.$$

Admission does not test parsing or acceptance. The size counts complete certificates, including annotations, binary fields and conversion witnesses. Decoding this finite family supplies one conservative rank, subject-size bound and finite label alphabet. The improved combinatory normalization results then cover every state along the chosen normalization of every decoded

accepted closed subject in a common finite syntactic carrier, retaining the original concluding type.

Those normalization and canonicity arguments are **external Rocq proofs**. Their rejection equations certify a finite table of the unchanged Bottom checker `raw_bottom_check`. For selectors $e_i : E_b$ naming all entries and this Bottom-checker table f_b, an independent finite-case guard is

$$\mathsf{Can}_b(x) = \Pi P : E_b \to U_2.\, P(e_0) \to \cdots \to P(e_{k-1}) \to P(x) : U_3.$$

With $\mathsf{Holds}(v) = v\, U_1\, \top\, \bot$, each branch of $P(x) = \mathsf{Holds}(f_b x) \to \bot$ converts to $\bot \to \bot$. Finite elimination gives an actual original-PTS term

$$c_b : \Pi x : E_b.\mathsf{Can}_b(x) \to \mathsf{Holds}(f_b x) \to \bot, \qquad \mathsf{type}(c_b) : U_4.$$

This handles arbitrary guarded inputs, not just canonical selectors. Quotation remains beta-injective at each bound. However, the carrier, table and branch proof are chosen externally from b.

Represented Church dictionaries remove the varying case telescope from the consumer: a separate All-package holds branch proofs, while Guard/Fill express independent membership and unused-branch filling. One internal eliminator consumes them. It does not construct All from an arbitrary represented budget. In particular, the identity cannot inhabit $\mathsf{Holds}(fx) \to \bot$ for symbolic f, x merely by unfolding a fold. The implemented dictionary parameter is $E : U_1$; integration with the raw carrier $C : U_2$ still needs an exact-rank adaptation.

`FiniteRawTables` supplies a simpler nondependent bridge:

$$d_b : E_b \to C, \qquad \mathsf{type}(d_b) : U_2.$$

The term re-abstracts the raw carrier's algebra and instantiates the finite selector only at its final $X : U_0$, never at $C : U_2$. It accepts arbitrary open bare selectors. Canonical inputs reduce forward to algebra-expanded quotations, proved beta-equivalent to the original quotations. Expanded quotation remains syntax-injective. These are beta laws for explicit constructors, not eta or parametricity for arbitrary inhabitants.

The separate `FiniteRawDecoding` route consumes independent $\mathsf{Enum}_2(x)$, the finite generation predicate above. Its constant motive $\lambda x.C$ gives a rank-four-type dependent eliminator whose canonical outputs reduce forward to the literal original quotations. Every raw tree, including malformed inputs, receives typed admission at its serialized size, with the native complete-input decoding roundtrip retained. The two decoders agree on canonical witnesses. Neither internally parses complete inputs, proves arbitrary-output coherence, or constructs its finite dictionary from an arbitrary represented budget.

3 Internal computation and represented resources

3.1 Comparisons retain their operands

`CheckerComparisons` compiles the unchanged checker into

$$e ::= \mathsf{Truth}(b) \mid \mathsf{Compare}(q, r) \mid \mathsf{Both}(e, e').$$

It covers all six rules, contexts, conversion instructions and malformed inputs. Comparison leaves contain full raw operands, not precomputed equality answers. Parsing, context lookup and computing endpoints, including substitution, currently occur during this external compilation.

`InternalCodeComparison` provides an independent traversal $\mathsf{Zip}(x, y) : U_4$ over the raw carrier $C : U_2$. Its clauses describe an atom versus arbitrary input, a pair versus an atom, and two pairs with corresponding child traversals. None contains a Boolean answer or rejection claim. One comparator of full-type rank five folds these clauses into actual tag tests and conjunctions. Its root test opens the Boolean answer type $X : U_0$ before folding C; it does not treat $\mathsf{Bool} : U_1$ as an admissible U_0 fold argument.

A rank-three conjunction-expression carrier and a fixed evaluator of full-type rank four execute those comparisons. On the externally compiled standard input, `CheckerExecution` proves exact forward reduction:

$$\mathsf{run}(\mathsf{compile}(q)) \twoheadrightarrow_\beta \mathsf{point}(b) \quad \Longleftrightarrow \quad \mathsf{rawCheck}(q) = b.$$

The Bottom checker has the analogous theorem. Both Boolean outcomes are retained. Raw serialization/quotation is faithful, but residual checking programs need not be: different sort certificates can yield the same program.

3.2 Joint traversal: checked interface and remaining construction

`InternalJointTraversal` represents aligned operands in a rank-four carrier, with actual raw projections and an independent rank-six generation guard. Its constructors and formation are checked. The proposed rank-seven-type operation turning guarded joint data into Zip evidence is not: large conversion checks exhausted resources. This is a proof-engineering limit, not a no-go theorem. Even that operation would leave alignment of independent inputs and source agreement to prove. The Horn route does not require this particular builder.

3.3 Storage bounds now include full operands

`CheckerResourceBounds` records both complete operands at every synchronized comparison node. For its raw traversal data $T(q, r)$,

$$|T(q, r)| \leq (3 + |q| + |r|) \min(|q|, |r|).$$

The residual-data encoding retains every expression constructor and operand; it is injective even though the executed program need not be. Exact size accounting and original-PTS capacity witnesses cover the source, expression data, every comparison operand and every recursive traversal payload. One externally constructed envelope covers a whole complete-code size-bounded fragment, including malformed and rejected sources.

These are **storage** results, not bounds on emitted PTS terms, reduction costs, or the compiler's intermediate allocations. The envelope itself is still generated externally. It does not assert that the stored expression is the checker program belonging to the stored source.

4 Source provenance and internal execution

4.1 Independent source graphs

Resource coverage does not determine a program's meaning. In `ComputationBoundary`, a scoped rejected source q has actual generation, capacity and traversal witnesses, while $\mathsf{Compare}(q, q)$ returns true. This refutes the resource-only agreement contract in Rocq, not the whole consistency programme or an internally quoted universal sentence.

`CheckerSourceGraph` repairs this contract **at the native level**. Its local inductive clauses record source fields, beta/equality endpoints, context lookup, all six typing-rule checks, context checks and both raw wrappers. Matching nodes retain the two actual operands and conjunction topology. No clause assumes acceptance, native typing or evaluator/checker agreement. Induction proves agreement with the unchanged compiler, uniqueness, coverage of every raw source, and both Boolean outcomes. Consequently, adding this graph to the independently generated storage package excludes the self-comparison counterexample.

Original substitution, context lookup, encoding and decoding in these clauses are still native operations. This graph is a specification with proved standard coverage, not yet its internal PTS representation.

4.2 One internal execution-agreement term

For the existing expression carrier $E : U_3$ and $\mathsf{Bool} : U_1$, `InternalExecutionGraph` independently represents the local clauses

$$\mathsf{Run}(\mathsf{leaf}(b), b), \qquad \frac{\mathsf{Run}(e, b) \quad \mathsf{Run}(f, c)}{\mathsf{Run}(\mathsf{both}(e, f), \mathsf{and}(b, c))}.$$

Here b, c are arbitrary Boolean programs, not necessarily canonical selectors. The clauses contain no evaluator equation. A Church fold with exact motive rank three gives $\mathsf{Run}(e, b) : U_5$. Define

$$\mathsf{Same}(b, c) := \Pi P : \mathsf{Bool} \to U_1.\, P(b) \to P(c) : U_3.$$

The checked original-PTS term

$$\mathsf{agree} : \Pi e : E.\Pi b : \mathsf{Bool}.\, \mathsf{Run}(e, b) \to \mathsf{Same}(\mathsf{evaluate}(e), b)$$

has full-type rank six and applies to **arbitrary represented execution witnesses**. Its proof folds the local clauses, using the evaluator's beta laws and predicate transport through conjunction. This is exact-rank Leibniz transport, not arbitrary-rank equality or beta equality.

Taking $P = \mathsf{Holds}$ yields an internal acceptance-transport term and one rank-six-type rejection operation

$$\Pi e : E.\, \mathsf{Run}(e, \mathsf{false}) \to \mathsf{Holds}(\mathsf{evaluate}(e)) \to \bot.$$

False execution must first be derived; it is not resource admission and does not itself establish source rejection.

`CheckerGraphRealization` constructs execution witnesses by recursion on residual syntax. Their outputs are the symbolic conjunctions of actual comparison programs, not precomputed checker answers. Applying the single internal agreement term and the native source graph gives exact standard realizations for both outputs. Ordinary accepted judgments receive actual inhabitants of residual acceptance; false outputs receive conditional rejection inhabitants. No external Bottom-emptiness argument is used. Raw quotation remains faithful; neither residual programs nor these execution witnesses are claimed to encode sources injectively.

4.3 Represented operations and the remaining boundary

Original context lookup now has a separate represented operation graph. Literal contexts are encoded by empty and $\mathsf{ext}(g, A)$; these are not certificate-context codes, which also store declaration proofs and ranks. The native erasure between them is explicit. The lookup clauses perform one annotated shift at **each** crossed extension, including index zero:

$$\frac{\mathsf{Shift}_{n,n+1}(A, A')}{\mathsf{Lookup}(n+1, \mathsf{ext}(g, A), 0, A')}, \qquad \frac{\mathsf{Lookup}(n, g, i, A) \quad \mathsf{Shift}_{n,n+1}(A, A')}{\mathsf{Lookup}(n+1, \mathsf{ext}(g, B), i+1, A')}.$$

The displayed successors abbreviate explicit represented successor edges. Lookup fibers have rank 18; a rank-19-type term derives output scope from an arbitrary lookup witness. Whole-context scoping is a separate rank-14 guard: the zero clause need not inspect its prefix. Executable standard witnesses follow the original `Context.index` exactly and retain injective context quotation. These facts do not prove arbitrary-output functionality, dependent typing, or internal certificate-context erasure.

For comparison, a proof-indexed relation $\mathsf{Trace}(x, y, z, b) : U_6$ retains the **actual** witness $z : \mathsf{Zip}(x, y)$. Its local clauses specify the corresponding Zip constructor and symbolic tag test or conjunction, not evaluator agreement. Formation and all three local Boolean-agreement cases are checked for arbitrary open inputs, with child agreement as induction data. The full constructor/induction assembly and its intended agreement term remain unverified. A separate executable symbolic-output compiler has the correct beta law for every raw operand pair, including malformed trees. Standard beta agreement is not a substitute for the missing arbitrary-trace theorem.

The represented source graph should retain context, certificate, subject, type and expression indices, sharing each chosen endpoint across its uses. Most endpoint clauses only assemble syntax. The substantive operations are:

- context lookup's endpoint law and its connection to certificate-context erasure, beyond the proved standard witnesses and output-scope law;
- annotated substitution for application and beta-contraction endpoints, with the coherence needed by their comparison consumers;
- product-rank computation and complete certificate/context parsing, including malformed failures.

Existing replacement evidence proves arbitrary-output scope preservation and realizes the exact standard substitution. It does not prove arbitrary endpoint coherence; its general environment can be multivalued. Restrict to the actual single-substitution environment and prove the consumer's needed law, not unconditional global functionality. Term scope alone is not full certificate parsing.

Thus native source agreement and internal residual execution agreement are both established, but their **internal source-indexed combination** is not. A total internal compiler remains

optional. Independently specified local relational traces suffice only after internal agreement and standard coverage are proved; inserting evaluator/checker agreement as a field would still assume the missing theorem.

5 Remaining milestones and acceptance gates

1. **Computation witnesses.** Residual execution laws are now internal. Prove the endpoint laws actually used by the source bridge. On the current checker route this includes arbitrary comparison-trace agreement and lookup/substitution coherence. Neither that particular implementation nor the proposed joint Zip builder is an unconditional prerequisite.
2. **Internal source bridge.** Translate independently admitted source proof nodes into the guarded Horn proof type, or finish the direct checker relation. Cover all six original typing rules, contexts, annotated substitution and supplied conversion instructions. Establish internal agreement and standard acceptance without inserting them as fields.
3. **Prove operational resource sufficiency.** Extend the checked storage envelopes to the actual operations, intermediate states and any proof transport performed. Supply standard raw-input admission independently of acceptance. Internal compression is needed only if the chosen resource construction uses compressed labels.
4. **Internal finite-fragment soundness.** Reuse the checked arbitrary-proof Horn consumer after the source bridge, with the required fragment and false-query interface. Alternatively derive bounded normalization or a sufficient negative continuation and exclude Bottom internally. Generate rejecting branches; do not assume an All-package.
5. **Assemble uniform bounded soundness.** Integrate the dictionary, checker and resource interfaces at fixed exact ranks and construct (BS) for arbitrary represented budgets.
6. **Eliminate resources and complete adequacy.** Construct $\mathsf{Cons}_{\mathrm{dir}}$, verify its original formation/typing, and discharge admission, acceptance realization and any translation to the intended historical sentence.

Resource work and the logical-scope audit can proceed alongside checker agreement and soundness. They are not reasons to postpone the hard local laws.

5.1 Reuse indexed Horn soundness

The combinatory development has finite total models of all six Hilbert rules and all 55 guarded MM0 axioms at size $N \geq 5$, depth $d \geq 3$, with unguarded Void denial. It now goes beyond standard satisfaction certificates: `IndexedHornSoundness` and `MM0IndexedReflection` provide actual original-PTS consumers for **arbitrary open queries and proof inhabitants**. Independent $\mathsf{Proof}(q) : U_3$ quantifies a predicate with rank-one fibers over the finite atom carrier and the positive clause handlers; it contains no model or source-typing premise. Consumer types have rank four, and the fixed-MM0 positive model premise is discharged. `IndexedHornTransport` maps proofs along literal atom maps with proved clause inclusion. `MM0InternalDecoder` instantiates this with semantic-to-raw observations. Supports are fixed before the query, not chosen from it.

Reuse this consumer instead of rebuilding internal dependent preservation when a source-to-Horn bridge suffices. The outstanding bridge must handle arbitrary admitted original-PTS proof codes, not just externally compiled proofs. Exact union coverage currently concerns the independent catalogue; original-PTS-to-catalogue completeness remains open. Queries are now internal; finite clause tables, supports and budgets remain external parameters. The Void specialization also retains external native canonicity in its certification. Thus the fixed-fragment

Horn core is checked, but milestone 4 for the original raw-source interface and an independent internal consistency proof are not complete.

5.2 A uniform clause interface, not yet a uniform model

`UniformHorn` removes the externally varying clause telescope from the generic proof algebra. Its actual PTS arguments are $D : U_1$, $A : U_a$ ($a = 1$ or 2), families $\mathsf{Cl}(d) : U_1$ and $\mathsf{Pos}(d, c) : U_1$, and projections $\mathsf{prem}(d, c, i), \mathsf{head}(d, c) : A$. Define

$$\begin{aligned}
\mathsf{Prem}_X(d, c) &:= \Pi i : \mathsf{Pos}(d, c).X(\mathsf{prem}(d, c, i)) : U_2, \\
\mathsf{Handler}_X(d) &:= \Pi c : \mathsf{Cl}(d).\mathsf{Prem}_X(d, c) \to X(\mathsf{head}(d, c)) : U_3, \\
\mathsf{Proof}(d, q) &:= \Pi X : A \to U_1.\mathsf{Handler}_X(d) \to X(q) : U_4.
\end{aligned}$$

One constructor combines represented child proofs. One consumer sends p to $p\,V\,h_V$; both have full-type rank five and a checked fold beta law. The fragment d, query, proof and all projection functions are genuine internal arguments. The two exact atom ranks are checked separately, not identified by cumulativity. The proof family contains no semantic premise; the consumer's interpretation V and handler h_V are supplied separately.

`UniformHornReindex` also binds a clause family $S(d) : U_1$ and a map $g_d : S(d) \to \mathsf{Cl}(d)$ internally. Source positions, premises and heads are literal pullbacks along g_d. A rank-five-type term transports arbitrary source proofs by replacing the handler with $\lambda s\, u.h(g_d(s), u)$. Interpretation after transport and composition of two such transports satisfy checked beta equations, without eta, injectivity, surjectivity or semantic assumptions. This permits proof movement between represented clause selections once their actual map is constructed; it does not supply original-source coverage.

Dependent clause soundness needs more than a bare field table. For example, $K(c) = \mathsf{Prem}_V(d, c) \to V(\mathsf{head}(d, c)) : U_3$ can use independent $\mathsf{Enum}_3(c)$ case evidence. `UniformGuardedHorn` provides the corresponding generic interface: its handler takes an explicit $G(d, c) : U_g$ witness before the children. At each exact $g = 3, 4$, proof fibers have rank $g + 2$ and constructor/consumer types rank $g + 3$, with checked open-input laws and a fold beta equation retaining the guard. Both ranks matter: Enum_3 has rank four for nonempty alphabets but rank three for the empty alphabet. There is no automatic lifting between them. Concrete guard families, coverage and interpretation handlers still need construction; the guard must not assert the desired soundness result.

The missing step is a **concrete instance**: represented clause and position carriers with their projections, independent source-node coverage, and a constructed interpretation handler valid for the represented resources. The rank-one fragment carrier has not been identified with the rank-two raw carrier or the higher-rank dictionary carrier. Abstracting these parameters does not construct finite models uniformly or establish (BS). The finite raw decoder and generic Horn interface are therefore complementary results, not yet a composed source-to-soundness proof.

If normalization is used instead, scope preservation of `Syntax.subst` must be strengthened to the annotated codomain/conversion coherence its consumer needs. Native normalization and `NativeCanonicity.normal_void_empty` do not supply that internal implication. Avoid a same-index motive quantifying over its own higher-rank graph proofs: `InvariantObstructions` rules out that rank pattern. Raw intermediate terms in a valid conversion certificate need not be typed or strongly normalizing; an untyped beta-loop is formalized. Normalize the accepted subject, not those intermediates; use arbitrary-term confluence or a negative transport law for the latter.

6 Simplifications and logical scope

The direct route needs neither $H_{<\omega}$ nor an EA derivation. It also does not inherently need a total budget selector, numerical capacity comparison, global lookup functionality, full constructor inversion, complete dependent typing reconstruction after every reduction, or a normal-form output in the admitted domain. Prove the weaker local invariant and coherence actually required by each consumer. Structural resources can replace numeral conversion.

The PHOAS self-interpreter's `GenericFold` supplies typed composition once an algebra is built; it does not prove that algebra's source-rule soundness. Decoding a code of Bottom is useful only after independently obtaining that code. No CPS theorem or rank-only self-interpreter is a prerequisite here.

Retain the supplied Zermelo revision's useful devices: complete-certificate rank compression, independent generation guards, guarded finite elimination, and continuation packaging of positive resources. Its proof-transport cost claims must be checked for this representation. From Pakhomov's construction[1], retain the proof-local use of finite resources and the distinction between possible internal consistency formulations. His transfer theorems are not axioms of the present development.

The weak local finite satisfiability results in the combinatory note[2] are probably highly relevant to investigating a non-Gödelian interpretation. They do not, by themselves, prove the required quantitative internal soundness or a failure of a derivability condition. Investigate Willard's stability framework[3] and its deduction-system dependence[4] as a separate bridge: his self-justifying extensions do not give a proof inside unchanged PTS-rk merely by analogy.

Run that audit now, not only after assembly. Identify the exact domain, proof predicate, available proof-combination operations and applicable diagonal/derivability conditions. Standard-instance coverage does not identify two internal consistency sentences; a conventional or historical claim needs a uniform internal translation. No escape from an applicable incompleteness theorem is asserted.

Check original formation/typing, computation laws and fresh transitive assumption reports; compare complete recursive kernel inventories with pristine upstream. The resource-only counterexample does not refute the entire programme.

[1] `https://arxiv.org/pdf/1907.00877v2`
[2] `../CombinatoryLogic-rk/CombinatoryPTS-rk.tex`
[3] `https://arxiv.org/pdf/1108.6330v8`
[4] `https://arxiv.org/pdf/1307.0150v2`